

Demeter

Documentation & User Guide

1. What is Demeter?

Demeter is a Chrome extension that lets you test how a website behaves when something goes wrong with its data. You can make a request slow, make it fail, or replace the real answer with your own. It all happens inside your browser, using a side panel. You don't need any extra software, and you don't need to touch the website's server.

2. Where can you use it?

- **While building a site (local or staging):** Test loading screens and error messages before the real backend is finished.
- **On live websites (QA testing):** Check how a deployed site reacts to a slow or broken server.
- **Safe to use:** Changes only happen in your own browser tab. The real server is never touched, and other users are never affected.

3. What can it do?

- **Pick which requests to change:** Match an exact web address, or use `*` as a wildcard (for example `*/api/users*`).
- **Add a delay:** Make a response wait a few seconds (for example 3 seconds) to check that loading indicators work.
- **Fake an error:** Return an error such as 404 or 500 to check that the site shows a proper error message.
- **Swap in your own data:** Replace the real response with your own JSON, including empty or broken data, to see if the site copes.
- **Save sets of rules (presets):** Save your rules under a name like "Slow Network" or "Error States", and switch between them any time.
- **See all requests (Logs):** Watch every request the page makes, and turn a real one into a fake with one click.
- **Turn everything off quickly:** One master switch pauses Demeter without deleting your rules.

4. How does it work?

1. You create a **rule** in the side panel: which address to catch, and what answer to give.
2. Demeter watches the requests the page makes.
3. When a request matches your rule, Demeter waits for the delay (if you set one) and gives back your fake answer.
4. If nothing matches, the request goes through normally.

5. Every request shows up in the **Logs** tab, marked as fake or real.

5. How to install

1. Get the project files and run `npm install`, then `npm run build`. This creates a folder called `dist`.
2. In Chrome, go to `chrome://extensions`.
3. Turn on **Developer mode** (top right).
4. Click **Load unpacked** and choose the `dist` folder.
5. Click the Demeter icon in the toolbar to open the side panel.

Tip: Choose the `dist` folder, not the main project folder. Chrome needs the folder that has `manifest.json` in it.

6. Quick start

1. Open the side panel and click **+ New Rule**.
2. Type the address to catch in **URL pattern** (for example `*/api/users*`).
3. Set a **Delay** or an error **Status**, then click **Save**.
4. Reload the page and use the site as normal.
5. Open **Logs** to confirm your request was caught.

7. Helpful tips

- **End your pattern with `*`.** Patterns must match the whole address, including anything after a `?`. Without the `*`, some requests get missed.
- **Search boxes send many requests.** Typing "cat" sends requests for "c", "ca", and "cat". Use a pattern that covers all of them.
- **Rules are first come, first served.** The rule at the top of the list is checked first.
- **Reload the page after changes.** Tabs opened earlier may not pick up the newest rules.
- **Use the master switch** to compare "with Demeter" and "without Demeter".

8. Limits to know about

- Demeter only catches normal website requests (`fetch` and XMLHttpRequest). It can't catch WebSockets or requests made by service workers.
- It changes fake responses only. It doesn't edit real requests.
- Rules apply to every site, so use specific patterns.
- Logs keep only the last 40 requests.
- There's no import/export of rules yet.

9. What we learned building it

- **Browser parts are kept separate.** A web page, an extension's helper scripts, its background worker, and its side panel each live in their own space and can only talk through messages.
- **Slowing or replacing requests safely takes care.** The page has to keep working even when a request is delayed or replaced.
- **Testing slow versus broken is different.** Watching an app under a long delay and under a total failure showed why good error handling on the screen matters.

10. What's next

- **Random failures:** Make a chosen percentage of requests fail (for example 25%).
- **Share your rules:** Export rules to a file so teammates can run the same tests.
- **Ready-made presets:** One-click profiles like "Slow Mobile 3G", "High Traffic", or "Database Down".
- **Record and replay:** Capture real responses and turn them into editable rules.
- **Deeper network testing:** Slow down or break the connection itself, beyond what the page can do on its own.